

Distributed Elevator Control System Using SocketCAN

SYSTEM DESIGN REPORT

1. Problem Description

Modern elevator systems use distributed controllers to manage passenger requests, elevator position, motor operation, door operation, and system diagnostics.

This project implements a software-only distributed elevator control system using **Linux SocketCAN, Python, and the virtual CAN interface vcan0**.

The elevator is divided into independent Electronic Control Units (ECUs). Each ECU performs a specific function and communicates with the other ECUs using standard CAN messages.

The system supports:

- Passenger destination floor requests
- Elevator position monitoring
- Motor control
- Door control
- Controller state management
- ECU heartbeat monitoring
- Fault detection
- Fault recovery
- Human Machine Interface (HMI)

The objective is to demonstrate a distributed CAN-based control system without requiring physical CAN hardware.

2. Functional Architecture

The system consists of six ECUs and one separate HMI application.

Main Functions

Passenger Request ECU

- Accepts the destination floor from the user.
- Sends the requested floor to the Controller ECU.

Position/Sensor ECU

- Simulates the elevator position.
- Provides the current floor.
- Provides position status information.

Controller ECU

- Receives passenger requests.
- Determines the required movement direction.
- Controls the motor and door.
- Maintains the elevator control state.
- Handles safety conditions.

Motor ECU

- Receives motor commands.
- Simulates motor operation.
- Reports motor status.

Door ECU

- Receives door commands.
- Simulates door opening and closing.
- Reports door status.

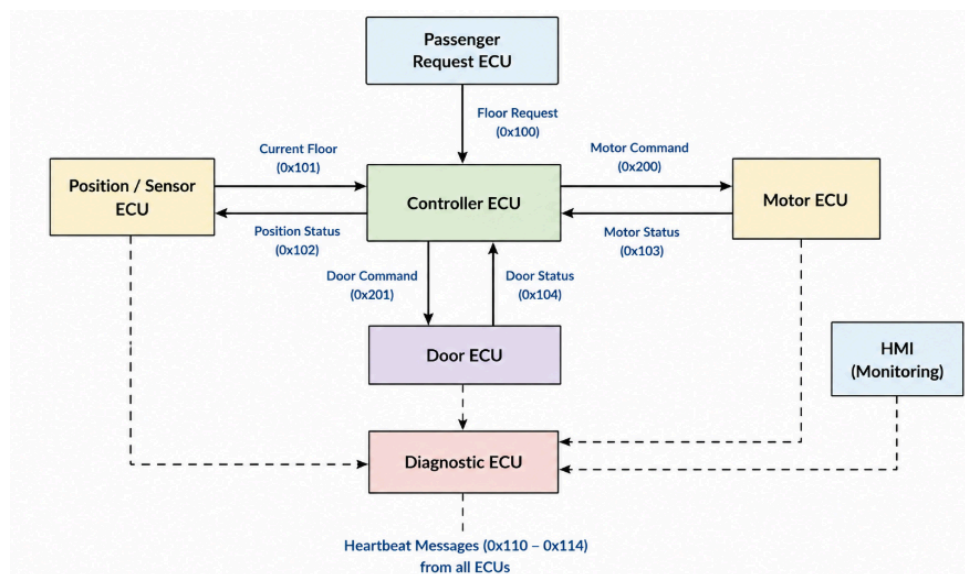
Diagnostic ECU

- Monitors ECU heartbeat messages.
- Detects communication failures.
- Detects invalid position information.
- Generates fault and recovery events.
- Maintains diagnostic logs.

HMI

- Displays the overall elevator system status.
- Shows current floor, requested floor, controller state, actuator status, ECU status, and diagnostics.

3. System Architecture



All ECU applications communicate through the Linux **SocketCAN vcan0** interface.

The HMI is a separate monitoring application and is not considered an ECU.

4. ECU Identification

ECU	Responsibility
Passenger Request ECU	Accepts destination floor requests
Position/Sensor ECU	Simulates elevator position and provides current floor
Controller ECU	Main elevator control logic and state management
Motor ECU	Receives motor commands and reports motor status
Door ECU	Receives door commands and reports door status
Diagnostic ECU	Monitors ECU communication, detects faults, logs events, and reports recovery

The corresponding source files are:

```
src/request_ecu.py
src/position_sensor_ecu.py
src/controller_ecu.py
src/motor_ecu.py
src/door_ecu.py
src/diagnostic_ecu.py
src/hmi.py
```

Common CAN communication functions are implemented in:

```
src/can_utils.py
```

5. CAN Network Design

The system uses the Linux SocketCAN virtual interface: vcan0

Standard CAN identifiers are used for communication between the ECUs.

CAN Matrix

CAN ID	Transmitter	Receiver	Message	Type
0x100	Passenger Request ECU	Controller ECU	Floor Request	Event-triggered
0x101	Position/Sensor ECU	Controller ECU / HMI	Current Floor	Periodic
0x102	Position/Sensor ECU	Diagnostic ECU / HMI	Position Status	Periodic
0x103	Motor ECU	Controller ECU / HMI	Motor Status	Periodic
0x104	Door ECU	Controller ECU / HMI	Door Status	Periodic
0x110	Passenger Request ECU	Diagnostic ECU	Heartbeat	Periodic
0x111	Position/Sensor ECU	Diagnostic ECU	Heartbeat	Periodic
0x112	Controller ECU	Diagnostic ECU	Heartbeat	Periodic
0x113	Motor ECU	Diagnostic ECU	Heartbeat	Periodic
0x114	Door ECU	Diagnostic ECU	Heartbeat	Periodic
0x200	Controller ECU	Motor ECU	Motor Command	Event-triggered
0x201	Controller ECU	Door ECU	Door Command	Event-triggered
0x202	Controller ECU	HMI / Diagnostic ECU	Controller Status	Periodic
0x300	Safety / Test Source	Controller ECU	Safety Command	Event-triggered

The system therefore uses both **periodic** and **event-triggered** CAN communication.

6. Signal Definitions

The main signals use unsigned 8-bit encoding.

Floor Request

CAN ID : 0x100
Signal : Destination Floor
Encoding : uint8
Range : 1-5
Type : Event-triggered

Current Floor

CAN ID : 0x101
Signal : Current Floor
Encoding : uint8
Range : 1-5
Type : Periodic

Position Status

CAN ID : 0x102
Signal : Position Status
Encoding : uint8
Type : Periodic

Value	Meaning
0	Unknown
1	Valid
2	Invalid
3	Position Sensor Fault

Motor Command

CAN ID : 0x200
Encoding : uint8
Type : Event-triggered

Value	Meaning
0	STOP
1	UP
2	DOWN

Door Command

CAN ID : 0x201
Encoding : uint8
Type : Event-triggered

Value	Meaning
0	CLOSE
1	OPEN

Motor status, door status, controller status, and heartbeat messages are also transmitted periodically.

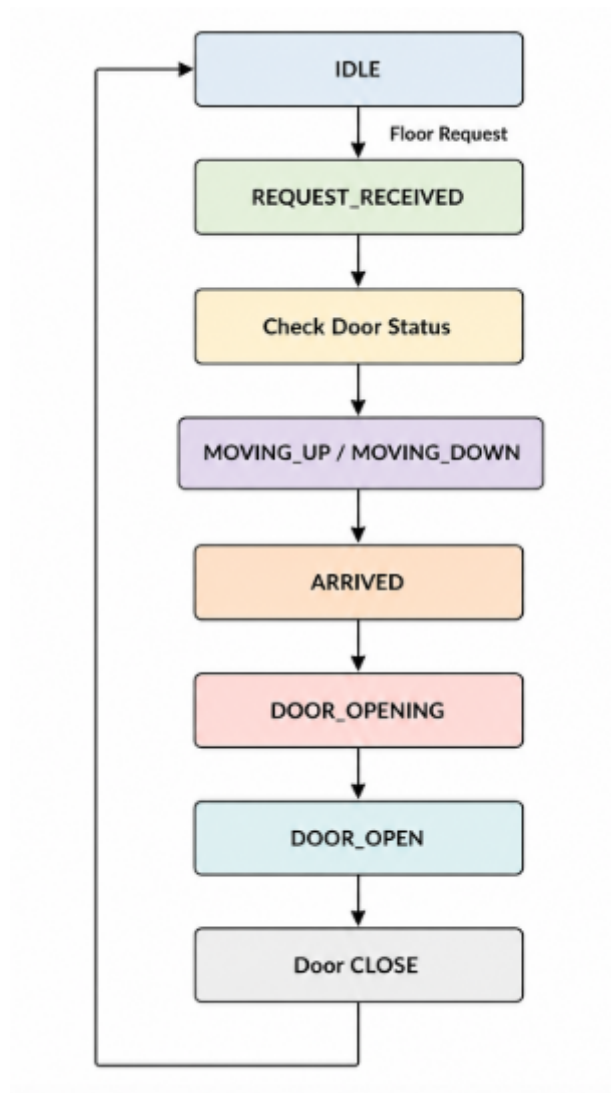
7. Controller ECU State Machine

The Controller ECU uses a state-based control strategy.

The main states are:

IDLE
REQUEST_RECEIVED
DOOR_CLOSING
MOVING_UP
MOVING_DOWN
ARRIVED
DOOR_OPENING
DOOR_OPEN
SAFE

The normal sequence is:



If a safety condition is detected, the Controller ECU enters the **SAFE** state and commands the motor to stop.

8. Normal System Operation

The normal elevator sequence is:

1. The passenger enters a destination floor through the Passenger Request ECU.
2. The Passenger Request ECU transmits the floor request using CAN ID `0x100`.
3. The Controller ECU receives the request.
4. The Controller ECU compares the requested floor with the current floor.
5. The Controller ECU commands the Motor ECU to move UP or DOWN.
6. The Position/Sensor ECU simulates elevator movement.
7. Current floor information is periodically transmitted.
8. When the target floor is reached, the Controller ECU commands the motor to STOP.
9. The Controller ECU commands the Door ECU to OPEN.
10. After the door operation, the door is commanded to CLOSE.
11. The controller returns to the `IDLE` state.

9. Human Machine Interface

A console-based HMI is implemented in: `src/hmi.py`

The HMI provides a consolidated system-level view.

It displays:

SYSTEM STATUS

Current Floor
Requested Floor
Controller State
Position Status

ACTUATORS

Motor Status
Door Status

ECU STATUS

Passenger Request
Position/Sensor
Controller
Motor
Door

DIAGNOSTICS

System Fault
Last Event

The HMI receives information from the CAN network and allows the overall system operation to be observed from a single terminal.

10. Diagnostics and Fault Handling

The Diagnostic ECU supervises the health of the distributed system.

It monitors heartbeat messages from:

Passenger Request ECU
Position/Sensor ECU
Controller ECU
Motor ECU
Door ECU

If an expected heartbeat is not received within the configured timeout period, the Diagnostic ECU reports a fault.

Example:

FAULT: Controller ECU heartbeat timeout

When communication resumes:

RECOVERY: Controller ECU heartbeat restored

When all monitored ECUs are operational:

SYSTEM RECOVERY: All monitored ECUs operational

The Diagnostic ECU also detects invalid position information.

Example:

FAULT: Invalid position status

Diagnostic events are recorded in the diagnostic log.

11. Safety and Fault Recovery

The system supports fault conditions including:

- Missing ECU heartbeat
- ECU/node failure
- Invalid position status
- Safety stop conditions

During a safety condition, the Controller ECU commands:

Motor Command = STOP

and enters: SAFE

When a failed ECU is restarted, heartbeat communication resumes and the Diagnostic ECU reports the recovery.

This provides basic fault detection and recovery behavior for the distributed system.

12. Design Decisions

The following design decisions were made for the implementation:

Distributed ECU Structure

Each ECU is implemented as an independent Python application. This separates responsibilities and represents the distributed architecture of an automotive/elevator control system.

SocketCAN

Linux SocketCAN was selected to provide CAN communication without requiring physical CAN hardware.

Virtual CAN

The `vcn0` interface allows all ECUs to communicate and be tested on a single Linux system.

CAN Message IDs

Dedicated CAN identifiers are assigned to commands, status information, and heartbeat messages to keep communication organized.

Heartbeat Monitoring

Periodic heartbeat messages allow the Diagnostic ECU to detect communication loss and ECU failures.

State-Based Control

The Controller ECU uses explicit states to make the elevator operation predictable and easier to verify.

Separate HMI

The HMI is implemented as a separate monitoring application so that system status can be observed without changing the ECU responsibilities.

13. Conclusion

The implemented system demonstrates a distributed elevator control architecture using Linux SocketCAN and Python.

Six independent ECUs communicate over `vcn0` to perform passenger request handling, position monitoring, motor control, door control, and diagnostics. A separate HMI provides a consolidated view of the system.

The implementation also demonstrates periodic CAN communication, event-triggered commands, heartbeat supervision, fault detection, safety handling, and recovery.